
MEMORA: A Harmonic Memory Representation

Balancing Abstraction and Specificity

Menglin Xia^{*1} Xuchao Zhang^{*1} Shantanu Dixit¹ Paramaguru Harimurugan¹ Rujia Wang¹ Victor Rühle¹
Robert Sim¹ Chetan Bansal¹ Saravan Rajmohan¹

Abstract

Agent memory systems must accommodate continuously growing information while supporting efficient, context-aware retrieval for downstream tasks. Abstraction is essential for scaling agent memory, yet it often comes at the cost of specificity, obscuring the fine-grained details required for effective reasoning. We introduce MEMORA, a harmonic memory representation that structurally balances abstraction and specificity. MEMORA organizes information via its *primary abstractions* that index concrete memory values and consolidate related updates into unified memory entries, while *cue anchors* expand retrieval access across diverse aspects of the memory and connect related memories. Building on this structure, we employ a retrieval policy that actively exploits these memory connections to retrieve relevant information beyond direct semantic similarity. Theoretically, we show that standard Retrieval-Augmented Generation (RAG) and Knowledge Graph (KG)-based memory systems emerge as special cases of our framework. Empirically, MEMORA establishes a new state-of-the-art on the LoCoMo and Long-MemEval benchmarks, demonstrating better retrieval relevance and reasoning effectiveness as memory scales.

1. Introduction

Large language models (LLMs) have substantially advanced the capabilities of autonomous agents in planning, tool use, and multi-step reasoning (Wang et al., 2024; Guo et al., 2024). However, intelligence is not just the ability to reason in the moment; it is the ability to learn and adapt over time—a

capability rooted in how experience is organized, abstracted, and reused. While current agents excel at atomic problem-solving, they remain effectively *stateless*, treating recurring tasks and user intents as isolated events (Yao et al., 2023; Wu et al., 2023). Without a principled mechanism to organize accumulated experience, agents are forced to repeatedly re-derive plans and reproduce redundant reasoning steps, leading to brittle performance and escalating token costs. As agents are increasingly deployed in real-world environments, this lack of structured, reusable memory has become the critical bottleneck, limiting their ability to support complex, long-horizon workflows (Milam & Gulli, 2025).

Scaling agent memory requires resolving a fundamental tension between abstraction and specificity. Existing designs typically collapse into one of two extremes. Many approaches favor specificity, either by storing raw interactions or document fragments (Xu et al., 2025; Lewis et al., 2021) or by extracting atomic facts from text (Chhikara et al., 2025; Nan et al., 2025). While detailed, these strategies suffer from fragmentation: raw logs overwhelm the agent with unstructured noise, while isolated facts stripped of their narrative context often fail to capture dependencies inherent in long-horizon tasks. Conversely, others adopt coarse abstractions, compressing experience into high-level summaries (Zhong et al., 2023; Li et al., 2025). While efficient, this approach strips away task-critical nuances (e.g., specific constraints, edge cases, or numeric details), rendering the memory insufficient for precise execution. This representational gap cripples retrieval: because memory lacks a structured link between high-level concepts and low-level details, agents cannot effectively navigate their own history. They are left choosing between retrieving a deluge of irrelevant facts or a vague summary that lacks actionable utility, ultimately failing to support robust long-horizon reasoning.

To address these limitations, we introduce **MEMORA**, a harmonic memory architecture that structurally balances abstraction and specificity. MEMORA organizes experience through a dual-layered representation that acts as navigational scaffolding over concrete content. At the core is the *primary abstraction*, which defines the canonical identity of

^{*}Equal contribution ¹M365 Research, Microsoft. Correspondence to: Menglin Xia <mollyxia@microsoft.com>, Xuchao Zhang <xuchaozhang@microsoft.com>.

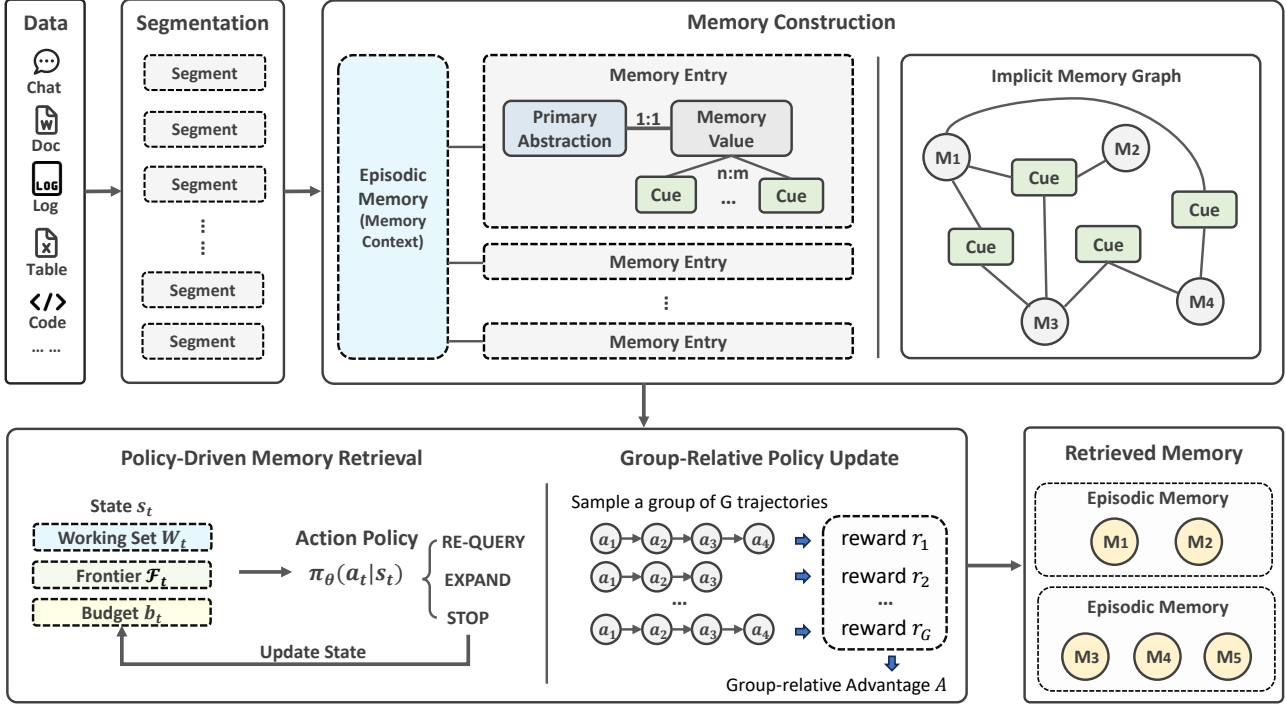


Figure 1. Overview of the MEMORA harmonic memory architecture.

a memory entry — capturing what the memory is fundamentally about. Each memory entry is composed of a primary abstraction paired with a memory value, where the value stores the specific memorized information. The primary abstraction acts as a coherent container, enabling MEMORA to incorporate emerging concepts as new entries while aggregating related updates into a unified record, thereby preventing conceptually related information from fragmenting into disjoint memory entries. For example, the evolving timeline of a project can be represented as a single memory entry under the primary abstraction *Project Memora Timeline*, within which milestones, design iterations, experiments, and decisions are incrementally appended. Complementing this, *cue anchors* are extracted from the memory value to serve as contextualized access points. By encoding diverse perspectives and aspects of a memory, these anchors expand retrieval access and establish a many-to-many connectivity across related memory entries. Together, this organization allows agents to navigate from concrete contexts to stable abstractions, supporting implicit relational reasoning and temporal coherence without the overhead of full-context processing.

Furthermore, we introduce a policy-guided retrieval mechanism that treats memory access as an active reasoning process. Retrieval is formulated over a discrete action space consisting of query refinement, memory expansion, and termination. By iteratively selecting these actions, the policy retriever refines the retrieved context to uncover relevant

information beyond immediate semantic similarity, effectively capturing multi-hop dependencies that static retrieval methods often miss.

Empirically, MEMORA establishes state-of-the-art performance on the LoCoMo and LongMemEval benchmarks (86.3% and 87.4% respectively), outperforming both strong memory baselines and full-context inference. Its ability to consistently outperform full-context inference demonstrates that memory retrieval guided by appropriate abstraction is more reliable than brute-force reconstruction for reasoning over extensive histories. By balancing abstraction with specificity, the harmonic organization of MEMORA provides a scalable foundation for long-horizon agent intelligence, reducing token consumption by up to 98% compared to full-context processing. The code is available at <https://github.com/microsoft/Memora>.

2. Related Work

Agentic Memory Management Systems Retrieval-Augmented Generation (RAG) (Lewis et al., 2021; Borgeaud et al., 2022; Gao et al., 2024) effectively extends the context capabilities of LLMs, but often lacks the precision required for long-horizon reasoning for agentic tasks. Consequently, recent research has shifted toward active memory management. Systems like MemGPT (Packer et al., 2023) draw inspiration from operating systems, introducing a virtual context management system that actively

swaps information between “active” context and archival storage. Similarly, MemOS (Chhikara et al., 2025), Memory OS (Kang et al., 2025) and MIRIX (Wang & Chen, 2025) propose architecture-level solutions for managing memory lifecycles. Other approaches focus on the mechanism of interaction: LangMem¹ treats memory as an external tool that agents explicitly call to update, while learning-based approaches like Mem-R1 (Yan et al., 2026) attempt to train models to manage their own memory policies autonomously.

Structured Memory Representations Parallel to management strategies, significant research has focused on how memory is represented and structured to improve organization and retrieval. Early attempts like MemoryBank (Zhong et al., 2023) utilized summarization to condense past events, while A-Mem (Xu et al., 2025) grouped memories into clusters. Mem0 (Chhikara et al., 2025) takes a different approach, prioritizing the lifecycle of factual memories with explicit mechanisms to add, update, and delete extracted facts. Nemori (Nan et al., 2025) attempts to combine episodic and semantic memory types to mirror human cognitive processes. However, without a cohesive structure, these isolated facts often become fragmented, leading to significant information loss during updates. Concurrently, graph-based representations, such as HippoRAG (Gutierrez et al., 2024), GraphRAG (Edge et al., 2025), Zep (Rasmussen et al., 2025), and Mem0-graph (Chhikara et al., 2025), have emerged to capture relationships between entities and support global reasoning. While graphs improve connectivity, they introduce distinct trade-offs: rigid schemas often abstract away critical details, while maintaining dense graph structures at scale can introduce significant retrieval noise. In addition, despite the structural innovations, the underlying representations often remain brittle, struggling to balance the specificity required for precision with the abstraction needed for scalability.

Positioning of MEMORA Existing memory systems often index and retrieve memories through their *content*, either as raw embeddings of extracted facts or via graph traversal over entity nodes. This couples *what* is stored to *how* it is accessed and forces the specificity–abstraction trade-off discussed above. MEMORA’s central novelty is to decouple these two concepts: memory content remains rich and unfragmented, while a separate structural layer of primary abstractions and cue anchors carries the navigational signal used for retrieval. Unlike graph-based methods, MEMORA also avoids the maintenance overhead of a pre-defined ontology: cue anchors are generated per memory and double as flexible metadata filters (source, timestamp, entity), so the same memory is reachable through multiple alternative access paths without committing to a fixed schema.

¹<https://langchain-ai.github.io/langmem/>

We provide a formal analysis demonstrating that MEMORA serves as a unified and strictly more expressive framework for memory retrieval. Traditional RAG and KG-based retrieval emerge as special cases under restricted configurations, while MEMORA supports richer mixed-key retrieval behaviors and principled efficiency improvements through abstraction-first scoping and structured traversal. More details including the proof can be found in Appendix D.

3. Method

We propose MEMORA, a harmonic memory representation designed to balance abstraction with specificity. We begin by formalizing the problem setting, followed by a detailed description of the proposed method.

3.1. Problem Formulation

We formulate memory management as the maintenance of a structured store derived from a continuous, heterogeneous data stream.

Let $\mathcal{D} = \{d_1, \dots, d_N\}$ denote a growing corpus of documents, logs, code, tables, or agentic interaction traces.

Our objective is to learn a memory construction function

$$\mathcal{F}_m : \mathcal{D} \rightarrow \mathcal{M},$$

that maps raw data to a structured memory set $\mathcal{M}$, and a retrieval function

$$\mathcal{Q}(q, \mathcal{M}) \rightarrow \mathcal{M}_q, \quad \mathcal{M}_q \subseteq \mathcal{M},$$

that, given a query q , selects a compact subset of relevant memory entries $\mathcal{M}_q$ to maximize downstream task utility.

The core design challenge is to maximize the relevance of $\mathcal{M}_q$ while minimizing its size ($|\mathcal{M}_q| \ll |\mathcal{M}|$) and retrieval latency, necessitating a representation that supports both high-level semantic scanning and fine-grained contextual lookup.

3.2. MEMORA Overview

Figure 1 illustrates the overall architecture of MEMORA. Raw data from multiple sources are first segmented into semantic units, each associated with episodic context capturing situational information. These segments are transformed into harmonic memory entries, where each entry consists of a primary abstraction paired with a memory value and augmented with cue anchors. Primary abstractions provide stable canonical identities that consolidate related and evolving information, while cue anchors induce many-to-many associations across memory entries. Based on shared cue anchors and abstraction-level relationships, these associations

give rise to an implicit memory graph that encodes relational structure among memory entries without requiring explicit edge construction. At query time, an agent query is jointly matched against primary abstractions and cue anchors to identify relevant memory entries. Memory reasoning then traverses the resulting abstraction- and cue-based associations to retrieve a coherent set of related memory entries together with their episodic contexts. This design enables scalable, context-aware retrieval that supports downstream reasoning, planning, and decision-making without requiring full interaction histories to be reconstructed in the context window. The retrieval policy can be further optimized using Group-Relative Policy Optimization, which trains the policy by comparing groups of retrieval trajectories and updating it based on relative advantages, encouraging effective multi-step navigation and early stopping behavior.

3.3. Segmentation

Given a data item $d \in \mathcal{D}$, we first apply a segmentation function $\mathcal{S}(d)$ to decompose the content x into a set of semantically coherent segments $\{s_1, \dots, s_k\}$. Each segment s_i serves as the input unit for memory construction. This segmentation step determines the granularity at which memory entries are created and updated, enabling primary abstractions to consolidate related information while preserving contextual specificity. Notably, a single segment may give rise to multiple memory entries. The implementation of $\mathcal{S}$ depends on the data format: we employ a prompt-based extraction mechanism for unstructured narratives, but leverage structural hierarchies (such as document headers) for formatted files.

3.4. Episodic Memory

Episodic memory in MEMORA captures the narrative context associated with each segment. For every segment s_i , we construct an episodic memory $e_i = \mathcal{E}(s_i)$ that serves as a shared narrative grounding for all memory entries derived from that source. Crucially, the representation of e_i is flexible: it can take the form of an extracted high-level summary—capturing participants, intent, and temporal scope—or retain the raw segment text itself to preserve exact phrasing and subtle cues. This design allows episodic memory to function as a contextual anchor, adapting the balance between compression and fidelity based on the domain.

During memory retrieval and reasoning, episodic memories play a central role in preserving narrative coherence across retrieved items. Memory entries associated with the same episodic memory are grouped together, allowing the agent to recover the broader context surrounding individual facts. This episodic grouping supports coherent multi-step reasoning, planning, and decision-making in downstream agent workflows.

3.5. Primary Abstraction

To prevent memory fragmentation, we introduce *primary abstraction* to organize memory around stable, semantically meaningful concepts rather than individual observations. A primary abstraction a canonically represents a core concept or action, capturing what the memory is fundamentally about and serving as the stable organizing unit of memory. It allows related information, such as recurring events or evolving entity states, to be consolidated under a single persistent entry rather than fractured across redundant records.

The construction of the memory entries along with the primary abstraction follows a two-stage process: extraction and consolidation. Given a new input segment s , we first induce a set of candidate memory entries, each consisting of a proposed abstraction and its concrete content:

$$\mathcal{F}_a(s) = \{m_i\}_{i=1}^N, \quad m_i = (a_i, v_i), \quad (1)$$

where a_i represents the primary abstraction and v_i denotes the corresponding memory value, which stores the concrete details. This step proposes *potential* new memories prior to verification against the existing store.

In the consolidation phase, we integrate these candidates into $\mathcal{M}$. For a new candidate memory entry m_i , we first retrieve top- k existing entries most similar to the induced abstraction a_i :

$$\mathcal{R}(a_i) = \text{TopK}_{m \in \mathcal{M}}(\text{sim}(a_i, a_m); k), \quad (2)$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity between the primary abstraction embeddings. We refine this set by filtering out candidates below a similarity threshold γ :

$$\mathcal{U}(a_i) = \{m \in \mathcal{R}(a_i) \mid \text{sim}(a_i, a_m) \geq \gamma\}. \quad (3)$$

Next, an LLM-based selection function $\mathcal{J}$ determines if the new candidate (a_i, v_i) refers to the same underlying concept as any retrieved entry in $\mathcal{U}(a_i)$:

$$m^*(a_i) = \mathcal{J}(a_i, \mathcal{U}(a_i)). \quad (4)$$

Here $\mathcal{J}(\cdot)$ returns the target memory entry $m^*(a_i)$ if a match is found, or $\emptyset$ if the abstraction a_i is a novel concept.

The final memory construction operation follows a create-or-update rule:

$$m_i = \begin{cases} \text{Update}(m^*(a_i), a_i, v_i), & m^*(a_i) \neq \emptyset, \\ \text{Create}(a_i, v_i), & m^*(a_i) = \emptyset. \end{cases} \quad (5)$$

When a match $m^*(a_i)$ is found, the $\text{Update}(\cdot)$ operation merges the new content v_i into the existing memory $m^*(a_i)$, potentially also refining its abstraction to reflect the aggregated information, yielding an updated abstraction a'_i . Otherwise, $\text{Create}(\cdot)$ initializes a new memory entry. This policy

ensures that each memory entry remains anchored to a single primary abstraction, while enabling new information semantically aligned with existing content to be incrementally incorporated. As a result, the system enriches existing concepts with new details where possible, establishing new abstractions only when necessary.

3.6. Cue Anchors

While primary abstractions provide stable and compact organization of memory, they are intentionally coarse and do not capture all task-relevant details needed for flexible retrieval. To address this limitation, MEMORA introduces *cue anchors*, which serve as lightweight, fine-grained semantic hooks that complement primary abstractions by exposing additional retrieval paths into memory.

Given a memory entry $m_i = (a_i, v_i)$ constructed in the previous step, cue anchors are generated to capture additional salient signals not explicitly represented by the primary abstraction. Formally, cue anchor generation is defined as

$$\mathcal{F}_c(a_i, v_i) = \{c_{ij}\}_{j=1}^{|C_i|}, \quad c_{ij} \in \mathcal{C}_i, \quad (6)$$

where the resulting set $\mathcal{C}_i$ contains the cue anchors associated with memory entry m_i . Each cue anchor represents a salient aspect, attribute, or contextual perspective of the memory content, formatted as a composite of a main entity/topic and a key aspect. Unlike primary abstractions, which define the canonical identity of a memory entry, cue anchors are non-exclusive and form a many-to-many mapping: a single memory entry may be associated with multiple cue anchors, and the same cue anchor may appear across multiple memory entries.

When new cue anchors are generated, we perform an existence check against the memory store. If an anchor already exists, we simply link memory entry to the existing instance; otherwise, a new anchor is instantiated. Conversely, when memory entries are removed or merged, the corresponding cue-memory links are also updated. Any cue anchor that loses all associations is automatically pruned, ensuring the cue anchors remain compact and non-redundant.

4. Policy-Guided Memory Retrieval

Standard retrieval methods, such as semantic search (Karpukhin et al., 2020), often fail to capture the multi-hop dependencies required for complex reasoning. To address this, we formulate memory retrieval in MEMORA as a Markov Decision Process (MDP) (Puterman, 2014). Unlike static semantic search, a policy-guided retriever actively navigates the memory structure to construct a compact yet informative memory set $\mathcal{M}_q$ under a finite budget.

Algorithm 1 Policy-Guided Sequential Retrieval

Require: Query q , memory system $\mathcal{S}$, policy π_θ , budget B , max steps T

- 1: Initialize $q_0 \leftarrow q, \mathcal{M}_0 \leftarrow \emptyset, b_0 \leftarrow B$
- 2: Initialize frontier $\mathcal{F}_0 \leftarrow \text{InitFrontier}(q_0, \mathcal{S})$
- 3: **for** $t = 0, 1, \dots, T - 1$ **do**
- 4: $s_t \leftarrow (q_t, \mathcal{W}_t, \mathcal{F}_t, b_t)$
- 5: Select $a_t \sim \pi_\theta(\cdot \mid s_t)$
- 6: **if** $a_t = \text{STOP}$ **or** $b_t \leq 0$ **then**
- 7: **break**
- 8: **end if**
- 9: $(\Delta\mathcal{W}_t, \Delta\mathcal{F}_t, q_{t+1}) \leftarrow \text{Apply}(a_t, s_t, \mathcal{S})$
- 10: $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup \Delta\mathcal{W}_t$
- 11: $\mathcal{F}_{t+1} \leftarrow \text{UpdateFrontier}(\mathcal{F}_t, \Delta\mathcal{F}_t)$
- 12: $b_{t+1} \leftarrow b_t - \text{Cost}(a_t)$
- 13: **end for**
- 14: $\mathcal{M}_q \leftarrow \mathcal{W}_t$
- 15: **return** Retrieved memories $\mathcal{M}_q$

4.1. Memory Retrieval Policy Formulation

To operationalize the retrieval process, we define a step-by-step procedure where an agent iteratively observes the current state and selects actions to refine its memory set. The overall process is outlined in Algorithm 1.

Given a query q and a retrieval budget B , the system state at step t is defined as

$$s_t = (q_t, \mathcal{W}_t, \mathcal{F}_t, b_t). \quad (7)$$

Here, q_t is the current query representation, which can be refined over time; $\mathcal{W}_t$ represents the working set of memory entries retrieved so far; $\mathcal{F}_t$ is the frontier, representing a set of candidate memories explicitly linked to items in $\mathcal{W}_t$ but not yet retrieved, allowing the agent to observe what is reachable; and b_t is the remaining retrieval budget.

At each step, the policy $\pi_\theta(a_t \mid s_t)$ selects an action a_t from three atomic retrieval-control operations: **REFINE**, **EXPAND**, and **STOP**. **REFINE** regenerates or reformulates the query when the policy determines that the current query is insufficient or misaligned. This allows the agent to pivot its search strategy to target alternative information relevant to the final answer. **EXPAND** expands the working set by selecting relevant memories from the frontier $\mathcal{F}_t$. This action directly grows the working set with new evidence. **STOP** terminates the retrieval process when sufficient information has been gathered.

Executing an action a_t triggers the transition:

$$\text{Apply}(a_t, s_t, \mathcal{S}) \rightarrow s_{t+1}. \quad (8)$$

The working set accumulates new retrieved results, and the frontier is updated to include the neighbors of these newly

retrieved items:

$$\begin{aligned}\mathcal{W}_{t+1} &= \mathcal{W}_t \cup \Delta\mathcal{W}_t, \\ \mathcal{F}_{t+1} &= \text{UpdateFrontier}(\mathcal{F}_t, \Delta\mathcal{F}_t).\end{aligned}$$

Simultaneously, the remaining budget is reduced according to the cost of the selected action:

$$b_{t+1} = b_t - \text{Cost}(a_t). \quad (9)$$

The retrieval process terminates when either the STOP action is selected or the budget is exhausted. The accumulated working set $\mathcal{W}_t$ is returned as the final retrieved memory context $\mathcal{M}_q$.

4.2. Group-Relative Policy Updates

The policy π_θ can be implemented in various ways, ranging from a prompt-guided LLM (zero-shot) to a fully trained retrieval model. While prompt-guided policies based on off-the-shelf models can be directly applied for memory retrieval, they often fail to optimally balance retrieval cost against information gain. In this paper, we also explore optimizing the retrieval policy via group relative policy updates (Shao et al., 2024).

We treat retrieval as a preference learning problem. Given a query q , we sample a group of G retrieval trajectories

$$\mathcal{T}_q \triangleq \{\tau^{(i)}\}_{i=1}^G, \quad \tau^{(i)} = \{(s_t^{(i)}, a_t^{(i)})\}_{t=0}^{T_i}. \quad (10)$$

using the current policy π_θ , optionally mixed with a reference policy for exploration.

A trajectory-level judge assigns a scalar score $J(\tau^{(i)})$ to each trajectory based on three criteria: (i) correctness of the final answer, (ii) information redundancy among retrieved memories, and (iii) retrieval cost.

To reduce variance and dependence on absolute scalar rewards, we compute group-relative advantages within each query group:

$$\tilde{A}^{(i)} = J(\tau^{(i)}) - \frac{1}{G} \sum_{i'=1}^G J(\tau^{(i')}). \quad (11)$$

This normalization yields zero-mean advantages within each group, improving robustness to score scaling and judge bias while encouraging relative improvement among trajectories generated for the same query.

The retrieval policy is updated to increase the likelihood of actions from trajectories with positive relative advantage:

$$\mathcal{L}_{\text{GR}}(\theta) = - \sum_{i=1}^G \tilde{A}^{(i)} \sum_t \log \pi_\theta(a_t^{(i)} | s_t^{(i)}). \quad (12)$$

To stabilize training and prevent policy drift, we optionally regularize the update with a KL constraint relative to a reference policy π_{ref} :

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{GR}}(\theta) + \beta \sum_t \text{KL}(\pi_\theta(\cdot | s_t) \| \pi_{\text{ref}}(\cdot | s_t)). \quad (13)$$

This formulation enables preference-based optimization under sparse supervision and aligns naturally with the MDP-based sequential retrieval framework.

5. Experiments

We conduct extensive experiments to evaluate the effectiveness of MEMORA on long-context reasoning tasks.

5.1. Experimental Setup

Datasets. We evaluate our method on two long-context and multi-session reasoning benchmarks. **LoCoMo** (Maharana et al., 2024) comprises extensive multi-turn dialogues averaging 600 turns ($\sim 20k$ tokens). It challenges models with diverse question-answer pairs spanning single-hop, multi-hop, temporal, and open-domain tasks, requiring the synthesis of information across long conversational histories. **LongMemEval** (Wu et al., 2024) is a comprehensive benchmark for evaluating long-term memory robustness. We use the LongMemEval_S split (115k context length), which contains 500 questions derived from user–assistant interactions to test reasoning over extreme context windows.

Baselines. We compare MEMORA against a diverse set of baselines representing current state-of-the-art approaches: (1) *Full Context* that feed the entire context history into the prompt. (2) *RAG* that chunks context history and retrieves top- k fragments ($chunksize = 500$ and $k = 3$). (3) *Memory Systems* including **HippoRAG**, **Zep**, **Mem0**, **LangMem**, and **Nemori**, which utilize various strategies for memory management.

Evaluation Metrics. We report the **LLM-as-a-Judge** score as our *primary metric*, as it best captures the semantic validity of the generated answers. To ensure fair comparison, we adopt the same evaluation templates from prior work to assess the correctness of the responses. Full evaluation setup is detailed in Appendix B. We report **BLEU** and **F1** scores as complementary metrics on the LoCoMo dataset to measure the verbatim overlap between answers and the ground truth.

Retrieval Configurations. We evaluate MEMORA using three retrieval mechanisms: (1) *Semantic Retriever* (S), retrieval based on semantic similarity; (2) *Policy Retriever* (P), retrieval guided by a prompt-based LLM agent; (3) *GRPO Retriever*, retrieval guided by a policy trained via GRPO. To accommodate the training requirements of the GRPO

Table 1. Performance comparison on the LoCoMo dataset. Results for Zep, LangMem and Nemori are reported from Nan et al. (2025). MEMORA (S) and MEMORA (P) denote the results obtained using the semantic retriever and policy retriever, respectively.

Method	Multi-hop			Temporal			Open-domain			Single-hop			Overall		
	BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM
Full Context	0.356	0.459	0.766	0.506	0.572	0.819	0.204	0.250	0.500	0.557	0.634	0.885	0.487	0.565	0.825
RAG	0.222	0.324	0.557	0.428	0.486	0.548	0.224	0.277	0.458	0.448	0.507	0.710	0.389	0.455	0.633
HippoRAG	0.039	0.060	0.390	0.013	0.022	0.224	0.033	0.068	0.510	0.037	0.101	0.587	0.032	0.075	0.471
Zep*	0.204	0.305	0.537	0.200	0.239	0.602	0.193	0.242	0.438	0.400	0.455	0.669	0.309	0.369	0.616
Mem0	0.236	0.326	0.624	0.420	0.489	0.660	0.153	0.206	0.500	0.376	0.433	0.677	0.346	0.411	0.653
LangMem*	0.325	0.415	0.710	0.409	0.485	0.508	0.264	0.328	0.590	0.436	0.510	0.845	0.400	0.476	0.734
Nemori*	0.319	0.417	0.751	0.502	0.577	0.776	0.193	0.258	0.510	0.515	0.588	0.849	0.456	0.534	0.794
MEMORA (S)	0.321	0.417	0.784	0.502	0.624	0.851	0.251	0.318	0.594	0.522	0.597	0.900	0.464	0.552	0.849
MEMORA (P)	0.337	0.428	0.787	0.500	0.623	0.866	0.246	0.308	0.594	0.521	0.597	0.918	0.466	0.553	0.863

Table 2. Performance comparison on LongMemEval.

Question Type	Full Context	Nemori	MEMORA(S)	MEMORA (P)
Context length	115k	3.7-4.8k	2.1k	2.9k
single-sn-preference	16.7%	86.7%	76.7%	83.3%
single-sn-assistant	98.2%	92.9%	76.8%	78.6%
temporal-reasoning	60.2%	72.2%	84.2%	89.5%
multi-session	51.1%	55.6%	73.7%	78.2%
knowledge-update	76.9%	79.5%	96.2%	97.4%
single-sn-user	85.7%	90.0%	97.1%	98.6%
Average	65.6%	74.6%	83.8%	87.4%

Table 3. Component build-up ablation on LoCoMo

Method	Multi	Temp	Open	Single	Overall
MEMORA w/o abstraction (= Mem0)	0.624	0.660	0.500	0.677	0.653
MEMORA (primary abstraction, no update)	0.727	0.801	0.542	0.844	0.795
MEMORA (primary abstraction, with update)	0.780	0.813	0.510	0.836	0.801
MEMORA (semantic retriever)	0.784	0.851	0.594	0.900	0.849
MEMORA (policy retriever)	0.787	0.866	0.594	0.918	0.863

variant, we employ two evaluation setups. For our main results and ablation studies, we evaluate the *Semantic* and *Policy* retrievers on the **full** LoCoMo and LongMemEval datasets. For the GRPO experiments, we partition the LoCoMo dataset into train and test splits. We report GRPO metrics exclusively on the test partition to quantify the specific gains from policy optimization.

Implementation Details. All experiments utilize GPT-4.1-mini as the LLM backbone for memory curation, answer generation, as well as prompt-based policy retrieval. To ensure reproducibility, we fix the generation seed to 42 across all runs. Prompts used for memory extraction are provided in Appendix A.

5.2. Results and Analysis

5.2.1. PERFORMANCE ANALYSIS

Table 1 presents the results on the LoCoMo dataset. Our best-performing configuration, MEMORA with the *Policy Retriever*, achieves a score of **0.863**, followed by the *Semantic Retriever* variant at 0.849. MEMORA demonstrates

superior performance across all four task categories, establishing a new state-of-the-art.

Notably, MEMORA surpasses the *Full Context* baseline (0.825). We attribute this result to Memora’s ability to reduce “context noise”. By filtering out irrelevant dialogue turns and presenting a crystallized memory structure, MEMORA prevents the dilution of the model’s attention mechanism, effectively proving that *curated* context leads to sharper reasoning than *complete* context. MEMORA also significantly outperforms strong baselines, including RAG (0.633), as well as other competitive memory systems such as Mem0 (0.653) and Nemori (0.794). This performance gap validates the utility of our harmonic structure. The Policy Retriever further amplifies these gains by leveraging cue anchors to actively navigate the memory graph, ensuring that contextually linked information is retrieved even when it is not semantically adjacent.

Table 2 presents the performance on the LongMemEval dataset, where our method consistently outperforms strong baselines, achieving an accuracy of 87.4%.

5.2.2. ABLATION STUDIES

To understand the contribution of each component in MEMORA, we conduct two complementary ablations. Table 3 builds the system up from a no-abstraction baseline to the full model, quantifying what each architectural component contributes; Table 4 then performs a leave-one-out study over the full system, varying retrieval policy, memory types, and granularity.

Removing the primary-abstraction layer essentially reduces MEMORA to Mem0: both extract factual memories from data, but Mem0 embeds and retrieves the extracted text fragments directly, whereas MEMORA decouples *what* is stored (the memory content) from *how* it is retrieved by indexing through a structural abstraction layer (primary abstractions and cue anchors), allowing more expressiveness in the stored content and avoiding fragmentation. Even with cue anchors

Table 4. Ablation studies on retrieval policy and memory granularity the LoCoMo dataset.

Method	Avg. Tokens	Multi-hop			Temporal			Open-domain			Single-hop			Overall		
		BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM	BLEU	F1	LLM
Policy Retriever																
Episodic (Segment) + Factual	8499	0.337	0.428	0.787	0.500	0.623	0.866	0.246	0.308	0.594	0.521	0.597	0.918	0.466	0.553	0.863
Episodic (Segment) only	6624	0.350	0.451	0.780	0.517	0.610	0.847	0.260	0.328	0.625	0.544	0.619	0.903	0.485	0.568	0.851
Episodic (Segment) + Factual w/o cue	8425	0.329	0.416	0.773	0.512	0.631	0.857	0.243	0.299	0.594	0.518	0.596	0.905	0.465	0.552	0.851
Episodic (Extracted) + Factual	4467	0.328	0.417	0.762	0.521	0.646	0.860	0.245	0.303	0.615	0.475	0.543	0.880	0.443	0.526	0.838
Factual only	1853	0.309	0.398	0.801	0.522	0.646	0.851	0.225	0.277	0.542	0.484	0.551	0.870	0.444	0.526	0.833
Semantic Retriever																
Episodic (Segment) + Factual	7683	0.321	0.417	0.784	0.502	0.624	0.851	0.251	0.318	0.594	0.522	0.597	0.900	0.464	0.552	0.849
Episodic (Segment) only	6042	0.349	0.450	0.773	0.506	0.599	0.832	0.260	0.325	0.615	0.539	0.614	0.899	0.480	0.563	0.844
Episodic (Segment) + Factual w/o cue	7628	0.338	0.434	0.780	0.511	0.635	0.854	0.253	0.316	0.604	0.516	0.589	0.900	0.466	0.553	0.850
Episodic (Extracted) + Factual	3958	0.315	0.406	0.755	0.523	0.646	0.857	0.224	0.282	0.573	0.477	0.542	0.875	0.441	0.522	0.831
Factual only	1647	0.309	0.402	0.791	0.526	0.647	0.847	0.210	0.265	0.531	0.481	0.546	0.857	0.442	0.523	0.823

and policy retrieval disabled, adding the primary-abstraction layer alone lifts the overall LLM-as-a-Judge score from 0.653 to 0.795. Adding update, cue anchors and policy retrieval then provides further gains. This decoupling also produces far fewer memory entries (344 for MEMORA vs. 651 for Mem0 per conversation on average), which in turn improves retrieval efficiency. Direct embedding of memory content tends toward two extremes: Mem0 produces fragmented, isolated entries that lose inter-memory coherence, while RAG encodes rich content into a single fuzzy vector, preserving expressiveness at the cost of retrieval precision. MEMORA’s abstraction layer sidesteps both extremes by storing expressive, unfragmented content while indexing through lightweight abstractions. Appendix E provides three case studies illustrating these dynamics.

Comparing the two major retriever backbones in Table 4, the policy retriever consistently outperforms the semantic retriever. Crucially, this advantage disappears when cue anchors are removed, rendering the policy retriever comparable to the semantic approach. This highlights that the improvement is not merely a consequence of increased complexity in the policy network, but rather stems from its capacity to leverage cue anchors for traversing the memory graph. By following these anchors, the system can navigate to relevant non-local contexts that a semantic search would miss.

We also examine the impact of context granularity. We observe a clear performance hierarchy correlated with the richness of the episodic context: the variant using raw segments as episodic memory (*Episodic (Segment) + Factual*) achieves the highest score (0.863), outperforming the extracted episodic memory (*Episodic (Extracted) + Factual*, 0.838) and the *Factual Only* variant (0.833). This trend confirms that while discrete facts provide a solid baseline, the “connective tissue” found in episodic memory is essential for grounding. Furthermore, factual and episodic memories are not redundant but complementary. Adding factual memory to the episodic-only baseline consistently improves overall performance, indicating that MEMORA succeeds by

Table 5. Latency on the LoCoMo dataset. *End-to-end Latency* refers to the full inference workflow for each query, while *Search Latency* measures the memory retrieval steps.

Method	End-to-end Latency (s)			Search Latency (s)			Avg Steps
	Mean	P50	P95	Mean	P50	P95	
Policy Retriever							
Episodic (S) + Factual	5.697	5.004	10.974	4.609	3.857	9.581	3.45
Episodic (E) + Factual	5.438	4.703	10.593	4.497	3.719	9.437	3.39
Factual only	4.653	3.940	9.388	3.969	3.279	8.495	3.36
Semantic Retriever							
Episodic (S) + Factual	1.062	1.016	1.487	0.235	0.221	0.256	1
Episodic (E) + Factual	0.958	0.908	1.336	0.232	0.221	0.260	1
Factual only	0.733	0.676	1.006	0.220	0.200	0.245	1

combining the structural clarity of factual details with the richer context of the episodes.

Finally, we note the trade-off between performance and memory size. While the full *Episodic (Segment) + Factual* variant yields the best results, greater context richness inevitably leads to a larger memory footprint. However, the *Factual-only* configuration remains a strong “lightweight” alternative, achieving a respectable score of 0.833 while significantly reducing the context load. This highlights MEMORA’s flexibility for either maximum contextual fidelity or efficiency, depending on resource constraints.

5.2.3. INFERENCE LATENCY ANALYSIS

Table 5 details the latency metrics. For latency evaluation, we report the mean, P50 and P95 wall-clock latencies. These metrics capture both end-to-end response generation and retrieval operations across the LoCoMo dataset, accounting for real-world API overhead. We report these metrics across three memory configurations: *Episodic (Segment) + Factual*, *Episodic (Extracted) + Factual*, and *Factual Only*, as they represent different memory sizes. The policy retriever incurs higher latency compared to the semantic retriever, primarily due to the sequential nature of the search process. On average, the policy retriever requires over three steps per query. Since each step involves a distinct LLM call to determine the next action, the search latency naturally scales

Table 6. Average memory construction time per conversation on LoCoMo.

System	Time (s)	Performance
Mem0	1350.9	0.653
MEMORA	1322.0	0.863
MEMORA (offset)	739.9	0.860

Table 7. Using smaller memory-construction model on LoCoMo.

Model	Retrieval	Multi	Temp	Open	Single	Overall
gpt-5.4-nano	Semantic	0.713	0.620	0.479	0.867	0.763
gpt-4.1-mini	Semantic	0.784	0.851	0.594	0.900	0.849
gpt-5.4-nano	Policy	0.773	0.879	0.625	0.893	0.851
gpt-4.1-mini	Policy	0.787	0.866	0.594	0.918	0.863

with the number of iterations.

5.2.4. MEMORY CONSTRUCTION ANALYSIS

Construction overhead. Table 6 reports the average memory construction time per conversation on LoCoMo together with the corresponding overall performance score. Before optimization, MEMORA’s construction cost (1322.0s) is comparable to Mem0 (1350.9s), but with substantially higher answer quality (0.863 vs. 0.653), showing that the performance gains thus arise from a better memory representation rather than additional compute. We further apply an optimization that predicts index offsets into the source material rather than generating full memory values, which reduces construction time to 739.9s, a 45% speedup with minimal loss in quality.

Robustness to smaller construction LLMs. A natural concern is whether MEMORA’s gains depend on a powerful LLM at construction time. We replace the default gpt-4.1-mini with the less capable gpt-5.4-nano during memory construction; results are shown in Table 7. First, while a weaker construction model does produce slightly lower-quality memories, even the worst-case (nano + semantic, 0.763) MEMORA still substantially outperforms baselines that use gpt-4.1-mini for memory construction, including Mem0 (0.653) and RAG (0.633). This indicates that MEMORA’s advantage stems from its structural design rather than reliance on a powerful construction model. Second, policy-guided retrieval effectively recovers the construction-quality gap: nano + policy (0.851) drops only 1.4% from mini + policy (0.863) and nearly matches mini + semantic (0.849), showing that one can build memories with a much cheaper model and recover quality through policy-guided retrieval.

Memory Update Analysis. MEMORA’s update mechanism merges a newly extracted memory into an existing entry when their primary abstractions exceed a semantic-similarity threshold. We use a default threshold of 0.80,

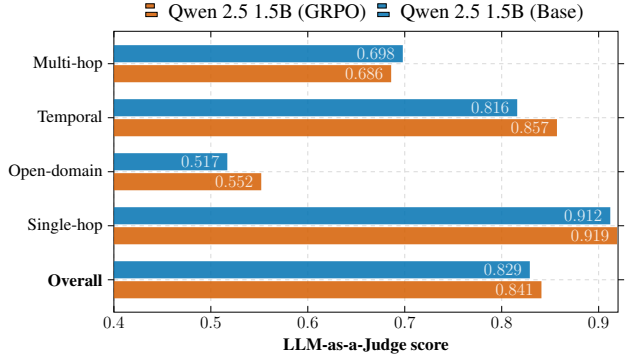


Figure 2. Results for GRPO training.

chosen to merge only entries that are clearly redundant while preventing over-aggressive consolidation. We provide an analysis on memory update sensitivity and how memory growth affects update frequency and construction overhead in Appendix F. We show that consolidation cost scales linearly with memory bank size and does not become a bottleneck as the memory store grows.

5.2.5. POLICY TRAINING

We further investigate whether the retrieval policy can be explicitly optimized using GRPO. We fine-tune a small backbone model (Qwen-2.5-1.5B) on a 70/30 train/test split of LoCoMo and evaluate on the held-out 30% test partition. As shown in Figure 2, the GRPO-tuned retriever achieves an overall LLM-as-a-Judge score of 0.841, consistently outperforming the base model baseline (0.829) across temporal, open-domain, and single-hop tasks. These results demonstrate that the retrieval policy is learnable and can be effectively distilled into smaller models, maintaining competitive performance compared to the instruction-tuned counterpart.

6. Conclusion

In this work, we introduce MEMORA, a harmonic memory architecture that balances abstraction and specificity for long-term agent memory. By introducing primary abstractions and cue anchors, MEMORA enables scalable, context-aware retrieval without fragmenting knowledge or obscuring task-critical detail. A policy-driven retrieval mechanism further allows agents to actively explore relevant memory beyond direct semantic similarity. We show that existing RAG- and KG-based memory systems arise as special cases of our framework. Empirically, Memora achieves state-of-the-art performance on long-horizon memory benchmarks, consistently outperforming strong baselines and full-context inference with both semantic and policy retrieval mechanisms, demonstrating the effectiveness of harmonic memory organization for scalable agent reasoning.

Impact Statement

This work advances the field of autonomous agents by enabling significantly more consistent and reliable long-term memory systems. By structurally balancing abstraction with specificity, MEMORA allows agents to retain and utilize context effectively over long horizons, addressing a key bottleneck in current architectures. This improvement in memory management paves the way for the development of a broader range of complex applications, from personalized long-term assistants to collaborative problem-solving systems, that require stable and precise context retention. To facilitate reproducibility and further innovation within the community, we release our code at <https://github.com/microsoft/Memora>.

References

- Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., de Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A., Brock, A., Paganini, M., Irving, G., Vinyals, O., Osindero, S., Simonyan, K., Rae, J. W., Elsen, E., and Sifre, L. Improving language models by retrieving from trillions of tokens, 2022. URL <https://arxiv.org/abs/2112.04426>.
- Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., and Larson, J. From local to global: A graph rag approach to query-focused summarization, 2025. URL <https://arxiv.org/abs/2404.16130>.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. Retrieval-augmented generation for large language models: A survey, 2024. URL <https://arxiv.org/abs/2312.10997>.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges, 2024. URL <https://arxiv.org/abs/2402.01680>.
- Gutierrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=hkujvAPVsg>.
- Kang, J., Ji, M., Zhao, Z., and Bai, T. Memory os of ai agent, 2025. URL <https://arxiv.org/abs/2506.06326>.
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and tau Yih, W. Dense passage retrieval for open-domain question answering, 2020. URL <https://arxiv.org/abs/2004.04906>.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- Li, Z., Song, S., Wang, H., Niu, S., Chen, D., Yang, J., Xi, C., Lai, H., Zhao, J., Wang, Y., Ren, J., Lin, Z., Huo, J., Chen, T., Chen, K., Li, K., Yin, Z., Yu, Q., Tang, B., Yang, H., Xu, Z.-Q. J., and Xiong, F. Memos: An operating system for memory-augmented generation (mag) in large language models, 2025. URL <https://arxiv.org/abs/2505.22101>.
- Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.
- Milam, K. and Gulli, A. Context engineering: Sessions & memory, 2025.
- Nan, J., Ma, W., Wu, W., and Chen, Y. Nemori: Self-organizing agent memory inspired by cognitive science, 2025.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Puterman, M. L. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X.,

- Wei, Z., and Wen, J. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), March 2024. ISSN 2095-2236. doi: 10.1007/s11704-024-40231-1. URL <http://dx.doi.org/10.1007/s11704-024-40231-1>.
- Wang, Y. and Chen, X. Mirix: Multi-agent memory system for llm-based agents, 2025. URL <https://arxiv.org/abs/2507.07957>.
- Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. Longmemeval: Benchmarking chat assistants on long-term interactive memory. 2024. URL <https://arxiv.org/abs/2410.10813>.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for llm agents, 2025. URL <https://arxiv.org/abs/2502.12110>.
- Yan, S., Yang, X., Huang, Z., Nie, E., Ding, Z., Li, Z., Ma, X., Bi, J., Kersting, K., Pan, J. Z., Schütze, H., Tresp, V., and Ma, Y. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning, 2026. URL <https://arxiv.org/abs/2508.19828>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memory-bank: Enhancing large language models with long-term memory, 2023. URL <https://arxiv.org/abs/2305.10250>.

A. Prompts for Memory Extraction

The following prompts were used to extract memories from conversation data:

```

You are an expert conversation segmentation specialist. Your goal is to analyze a
series of messages in a conversation and segment them into coherent topical episodes.

# TASK
Read the conversation carefully and identify points where the topic shifts
significantly.
Group messages discussing a similar subject, event, or theme into a single episode.
An episode is defined as a sequence of messages that revolve around a core topic or
theme.
Your task is to segment the conversation into such episodes.

# OUTPUT FORMAT
Provide a JSON object with the following structure:
{
  "episodes": [
    {
      "topic": "<brieftopicdescription>",
      "indices": [<listofmessageindicesinthisepisode>]
    },
    ...
  ]
}

Where each episode contains:
- topic: A brief description (a few words) summarizing the main topic of the episode
- indices: A list of 1-based indices of messages that belong to this episode

# GUIDELINES
1. Segmentation Criteria
  - Topical shift: Identify when a new subject, event, or theme is introduced.
  - Transitions: Look for phrases like "By the way", "Changing the subject", or "On
another note".
  - Time gaps: Large time lapses may indicate a new episode.
  - Setting changes: Changes in speaker, location, or context can signal a new
episode.
  - Topical grouping: Consecutive messages discussing the same topic belong to the
same episode.
2. Episode Length
  - Typically 2--8 messages per episode.
  - Combine messages if they discuss the same topic.
  - Avoid episodes longer than 8 messages covering multiple sub-topics.
  - Do not treat a single message as an episode unless it clearly marks a shift.
  - When in doubt, split into smaller episodes.
3. Formatting Rules
  - Use 1-based indexing for message indices.
  - Include all messages exactly once (no gaps or overlaps).
  - Indices in each episode should be consecutive.

# EXAMPLE OUTPUT
...
# CONVERSATION TO SEGMENT
{messages}

```

Figure 3. Prompt for segmenting conversations into coherent episodic units.

```
You are an expert episodic memory generator that creates episodic memory summaries
from conversation segments.

# TASK
Generate an episodic memory with an index and a detailed summary based on the provided
conversation segment.

Use the following format:
EpisodicIndex: [6--8 word summary capturing main topic, entity, or event]
EpisodicValue: [1--3 sentences descriptive summary of the conversation]

# GUIDELINES
1. EpisodicIndex
  - Create a short index (6--8 words) capturing the main topic or event of the
  episode.
  - Include specific context (e.g., domain or entity) to avoid vagueness.
2. EpisodicValue
  - Generate 1--3 sentence summary capturing:
    * Main information of the conversation segment (topic, theme, or event).
    * Relevant participants, referred to by name if available.
    * Use original wording when possible.
  - Focus on ``what happened`` rather than specific granular details.
  - Make the summary self-contained and understandable without the original
  conversation.
  - Include visual content if images are present.
  - Use only information present in the conversation segment; do not add external
  knowledge or infer beyond the content.

# INPUT
{content}

# OUTPUT
Provide the episodic memory in the format specified above.
```

Figure 4. Prompt for generating episodic memories from conversation segments.

```

You are an expert factual memory extraction assistant. Your goal is to extract
factual memories from a conversation segment.

# TASK
Read the input conversation carefully and extract ALL factual memories that could be
useful for future reference.

Produce each memory as a key-value pair in the following format:
MemIndex: memory index for retrieval
MemValue: memory value with all details supported directly from the given text.

# GUIDELINES
1. Content and Scope
  - Use only information explicitly mentioned in the conversation.
  - Capture ALL factual information that could be useful. When in doubt, create
    more rather than fewer memories.
  - Exclude greetings, small talk, or filler.
  - Split distinct facts into separate entries.
  - Include details about people, events, intentions, hobbies, preferences, states,
    beliefs, goals, future plans, times, and locations if mentioned.
  - Include visual content from images as textual context, integrating relevant
    facts naturally.

2. Format and Style
  - MemIndex: Short, human-readable, self-contained, unambiguous phrase. Include
    specific context (e.g., entity or domain) to avoid vagueness.
  - MemValue: One or two full factual sentences capturing all relevant details.
    * Use neutral and factual wording.
    * Use original wording from the conversation when possible.
    * Replace pronouns with specific names or entities for clarity.
    * Convert relative times/dates (e.g., ``yesterday``, ``next week``) to absolute
    dates based on the conversation timestamp.

Timestamp of conversation: {timestamp}
Input Conversation: {content}

# OUTPUT
Produce all factual memories in the format specified above.

```

Figure 5. Prompt for extracting factual memories from conversation segments.

```

You are a memory management assistant. Given a new memory entry and similar existing
entries, determine whether to update an existing entry or add a new one.

NEW MEMORY ENTRY:
Index: {new_index}
Value: {new_value}

EXISTING SIMILAR ENTRIES:
{candidates-info}

INSTRUCTIONS:
1. Analyze if the new entry should update any existing entry based on semantic
   similarity and content overlap.
2. If an update is needed, determine which candidate entry is best to update.
3. Generate an updated memory value that combines relevant information from both
   entries.
4. Decide whether the memory index should be updated to better reflect the combined
   information.

```

Figure 6. Prompt for deciding whether to update an existing memory entry or create a new one.

```

You are a memory-indexing assistant optimized for knowledge retrieval. Your goal is
to create cue indices that serve as semantic anchors for specific memories.

# TASK
For each memory provided, generate 1--3 short, meaningful CUE ANCHORS that can later
help recall or reason about that memory.
Provide the cue anchors as a list of strings for each memory.

# GUIDELINES
1. Definition: A cue anchor is a concise phrase (2--4 words) that anchors a specific
topic to a memory.
It follows the structure: [Main Entity] + [Key Aspect].
- Main Entity: the primary person, domain, or object involved (the ``who'' or
``what'').
- Key Aspect: the associated event, preference, action, state, or object.
Example patterns:
- [Person] [Event/Activity] → ``Jane hiking trip'', ``Mike vacation''
- [Person] [Hobby/Preference] → ``Michael jazz music'', ``Sophie vegan diet''
- [Person] [Condition/State] → ``Emma career change'', ``Liam health problems''
- [Person] [Object/Relation] → ``Alice research paper'', ``David guitar''
- [Domain] [Attribute/Artifact] → ``Project Orion timeline'', ``Product X
features''

2. Specificity: Avoid generic single words (e.g., ``summer'', ``happiness'',
``project meeting'').
Every cue anchor must be contextually anchored to a main entity mentioned in the
memory.
Use concrete aspects (e.g., ``Mike mental health problems'' rather than ``Mike
feelings'').

3. Atomicity: Each cue index should capture a single, indivisible aspect.
Do not include timestamps, exact numbers, or multiple descriptors.
Prefer generalizable cues (e.g., ``Mike birthday party'' over ``Mike birthday
party 2023'').

4. Distinct Facets: A memory may have multiple cue indices, each targeting a
different dimension.
Cue indices for the same memory should not overlap in meaning.
Avoid near-duplicates (e.g., ``Project Phoenix kickoff'' vs. ``Project Phoenix
launch'').

5. Uniqueness: Do not repeat the primary memory index as a cue index.

6. Purpose: Cue indices provide additional semantic keys beyond the primary index,
enabling recall, reasoning, and linking of related memories.

# EXAMPLES
Primary Abstraction: ``Jane's hiking trip to Appalachian Trail''
Memory Value: ``Last summer, Jane went on a week-long hiking trip along the
Appalachian Trail. She enjoyed the scenic views and challenging trails.''
Cue Anchors: [``Jane hiking'', ``Appalachian Trail views'', ``Jane summer trip']

Primary Abstraction: ``Mike's surprise birthday party''
Memory Value: ``Mike's friends organized a surprise birthday party for him at his
favorite restaurant Bistro Max.''
Cue Anchors: [``Mike birthday party'', ``Mike favorite restaurant'', ``Mike friends
gathering']

Primary Abstraction: ``Project Orion launch delay''
Memory Value: ``The launch of Project Orion has been delayed due to unforeseen
technical issues that need to be resolved.''
Cue Anchors: [``Project Orion launch'', ``Project Orion technical issues']

Primary Abstraction: ``Emma went swimming''
Memory Value: ``Emma went swimming during her vacation.''
Cue Anchors: [``Emma swimming']

# MEMORIES TO PROCESS
{memories}

```

Figure 7. Prompt for generating cue indices as semantic anchors for memory retrieval.

B. Evaluation Setup

Following prior work, we adopt the same evaluation protocol for LLM-as-a-judge scoring from prior work. Specifically, for LoCoMo, we use ANSWER_PROMPT from the official Mem0 GitHub repository <https://github.com/mem0ai/mem0/blob/main/evaluation/prompts.py> for answer generation, and https://github.com/mem0ai/mem0/blob/main/evaluation/metrics/llm_judge.py for LLM-as-a-judge scoring.

For LongMemEval, we use the evaluation prompt provided in the official GitHub repository https://github.com/xiaowu0162/LongMemEval/blob/main/src/evaluation/evaluate_qa.py for LLM-as-a-judge scoring.

To ensure a fair comparison, we employ gpt-4o-mini as the evaluation model across all experiments, consistent with prior work. Additionally, we fix the random seed to 42 for reproducibility.

For latency evaluation, we use a compute instance located in East US (32 cores, 128 GB RAM, 256 GB disk) and query an Azure OpenAI endpoint located in Sweden Central.

C. Preference-based Group-Relative Policy Updates.

C.1. Motivation.

In sequential memory retrieval, step-level rewards are often noisy or unavailable, while the true objective—such as answer quality, grounding, and efficiency—is typically observable only after completing an entire retrieval trajectory. Preference-based learning avoids explicit per-step supervision by comparing multiple retrieval trajectories generated for the same query and updating the policy to favor higher-quality trajectories.

C.2. Trajectory Generation.

Given a query q , we sample a group of G retrieval trajectories:

$$\{\tau^{(g)}\}_{g=1}^G, \quad \tau^{(g)} = \{(s_t^{(g)}, a_t^{(g)})\}_{t=0}^{T_g}, \quad (14)$$

using the current policy π_θ , or a mixture with a reference policy for exploration. Each trajectory produces a retrieved memory set $\mathcal{W}^{(g)}$.

C.3. Judge-Based Trajectory Scoring.

Each trajectory is evaluated by a judge that outputs a scalar score reflecting retrieval quality. The judge may be implemented as a lightweight learned model, a frozen LLM-based evaluator, or a deterministic heuristic. The trajectory score is decomposed into the following components.

Groundedness. Groundedness measures whether the final answer or reasoning is supported by the retrieved memories:

$$\text{Ground}(\tau) = \text{JUDGE}_{\text{ground}}(q, \mathcal{W}). \quad (15)$$

This term can be instantiated using LLM-based judgments of evidence support or heuristic measures such as entailment or citation coverage.

Redundancy. Redundancy penalizes repeated or highly overlapping memories:

$$\text{Redund}(\tau) = \frac{1}{|\mathcal{W}|^2} \sum_{m_i, m_j \in \mathcal{W}} \mathbb{I}[\text{sim}(m_i, m_j) > \delta]. \quad (16)$$

Cost. Cost accounts for retrieval budget consumption:

$$\text{Cost}(\tau) = \sum_t \text{Cost}(a_t). \quad (17)$$

C.4. Scalar Trajectory Score.

The judge aggregates the above components into a single trajectory-level score:

$$J(\tau) = w_1 \cdot \text{Ground}(\tau) - w_2 \cdot \text{Redund}(\tau) - w_3 \cdot \text{Cost}(\tau). \quad (18)$$

This score is defined at the trajectory level and does not require step-wise annotations.

C.5. Group-Relative Advantage.

Rather than relying on absolute scores, which may be noisy or query-dependent, we compute group-relative advantages within each query group:

$$\tilde{A}^{(g)} = J(\tau^{(g)}) - \frac{1}{G} \sum_{g'=1}^G J(\tau^{(g')}). \quad (19)$$

This normalization yields zero-mean advantages within each group, improving robustness to judge bias and score scaling while encouraging relative improvement.

C.6. Policy Update.

The policy is updated to increase the likelihood of actions from trajectories with positive relative advantage:

$$\mathcal{L}_{\text{GR}}(\theta) = - \sum_{g=1}^G \tilde{A}^{(g)} \sum_t \log \pi_{\theta}(a_t^{(g)} \mid s_t^{(g)}) . \quad (20)$$

To prevent policy drift, we optionally add KL regularization with respect to a reference policy π_{ref} :

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{GR}}(\theta) + \beta \sum_t \text{KL}(\pi_{\theta}(\cdot \mid s_t) \parallel \pi_{\text{ref}}(\cdot \mid s_t)) . \quad (21)$$

D. A Unifying Theory of Structured Memory Retrieval

D.1. Preliminaries and Notation

We briefly summarize the minimal notation required for theoretical analysis, relying on the definitions introduced in the Method section.

Let $\mathcal{M}$ denote the set of memory entries maintained by the system. Each memory entry is associated with a unique *primary abstraction* and a (possibly empty) set of *cue anchors*. We denote the primary abstraction space by $\mathcal{A}$ and the cue anchor space by $\mathcal{C}$.

The memory structure is characterized by two assignment relations:

$$\alpha : \mathcal{M} \rightarrow \mathcal{A}, \quad \Gamma : \mathcal{M} \rightarrow 2^{\mathcal{C}},$$

where $\alpha(m)$ assigns each memory entry m to exactly one primary abstraction, and $\Gamma(m)$ returns the set of cue anchors associated with m . These relations induce abstraction–memory and cue–memory associations, which together define the indexing structure over $\mathcal{M}$.

Given a query q , the system scores abstractions and cue anchors using query-dependent scoring functions

$$s_A(q, a), \quad s_C(q, c),$$

and selects a bounded set of top-ranked abstractions and cues. Retrieval is then defined structurally as the union of memory entries supported by the selected abstractions and cue anchors. This abstraction-and-cue–based retrieval operator constitutes the core retrieval mechanism analyzed in this section.

To support multi-hop and graph-style retrieval, memory entries may additionally be connected through traversal relations induced by shared cue anchors or other structural links. Let $\mathcal{R}_L(q)$ denote the result of applying up to L traversal steps starting from the initial retrieval set. Setting $L = 0$ recovers single-step retrieval, while larger L enables iterative expansion analogous to graph neighborhood search.

D.2. Traditional RAG and KG Retrieval as Special Cases

We show that both *traditional RAG* and *knowledge-graph (KG) retrieval* can be expressed as special cases of Memora by choosing appropriate key spaces and relations.

Theorem D.1 (Flat RAG as a Special Case of Memora). *Let $\mathcal{D}$ be a corpus and let $\mathcal{S}(\cdot)$ be a segmentation function that produces a set of chunks (segments). Consider a flat RAG retriever that, for any query q , returns*

$$\mathcal{R}_{\text{RAG}}(q) = \text{TopK}_{s \in \bigcup_{d \in \mathcal{D}} \mathcal{S}(d)} \text{sim}(q, s), \quad (22)$$

where sim is a similarity function over chunk representations. Then there exists a Memora instantiation and a policy π such that the retrieval set returned by Algorithm 1 equals $\mathcal{R}_{\text{RAG}}(q)$ for all queries q .

Proof. Define the Memora memory corpus by taking each chunk s as one memory entry,

$$\mathcal{M} = \{m(s) \mid s \in \bigcup_{d \in \mathcal{D}} \mathcal{S}(d)\}, \quad m(s) = (a(s), v(s), \mu(s)). \quad (23)$$

Let the primary abstraction equal the memory content,

$$a(s) = v(s) = s, \quad (24)$$

and let the cue-anchor set be empty for every entry,

$$\mathcal{C}(m(s)) = \emptyset. \quad (25)$$

Consider the restricted action space $\mathcal{A} = \{\text{QUERYA}, \text{STOP}\}$ and define the retrieval primitive QUERYA to return the top- k memory entries ranked by abstraction similarity,

$$\text{QUERYA}(q) = \text{TopK}_{m(s) \in \mathcal{M}} \text{sim}(q, a(s)) = \text{TopK}_{s \in \bigcup_{d \in \mathcal{D}} \mathcal{S}(d)} \text{sim}(q, s). \quad (26)$$

Let the policy π choose QUERYA at $t = 0$ and then STOP:

$$\pi(a_0 = \text{QUERYA} \mid s_0) = 1, \quad \pi(a_1 = \text{STOP} \mid s_1) = 1. \quad (27)$$

Algorithm 1 therefore terminates after one retrieval step and returns

$$\mathcal{W}_1 = \text{QUERYA}(q) = \mathcal{R}_{\text{RAG}}(q), \quad (28)$$

which proves the claim. $\square$

This theorem shows that flat chunk-based RAG corresponds to a degenerate configuration of Memora in which each segment forms a single memory entry, abstractions coincide with raw memory content, cue anchors are unused, and retrieval reduces to a single abstraction query step.

D.2.1. KNOWLEDGE GRAPH RETRIEVAL

We analyze the relationship between Memora and KG-based retrieval under two settings: (i) implicit KGs, where neighborhood structure is induced by semantic similarity, and (ii) explicit KGs, where symbolic relations are available. Both can be expressed within the Memora framework using cue anchors and traversal actions.

Implicit KG retrieval. We first consider KG-style retrieval without explicit relational edges. Let $\mathcal{M}$ be the memory corpus and let

$$\pi : \mathcal{M} \rightarrow V$$

associate each memory entry with an entity $v \in V$. Given a query q , an implicit KG retriever selects a seed entity set $S(q) \subseteq V$ and retrieves memories attached to entities reachable within L steps under a similarity-induced neighborhood relation.

Formally, define an implicit entity adjacency

$$v \sim v' \iff \text{sim}(v, v') \geq \delta,$$

and let $\text{Nbr}_L^{\text{imp}}(S(q))$ denote the L -hop neighborhood under this relation. The retrieval result is

$$\mathcal{R}_{\text{KG}}^{\text{imp}}(q) = \{m \in \mathcal{M} \mid \pi(m) \in \text{Nbr}_L^{\text{imp}}(S(q))\}.$$

Theorem D.2 (Implicit KG Retrieval as a Special Case of Memora). *For any implicit KG retriever $\mathcal{R}_{\text{KG}}^{\text{imp}}(q)$, there exists a Memora instantiation and traversal depth L such that $\mathcal{R}_L(q) = \mathcal{R}_{\text{KG}}^{\text{imp}}(q)$ for all queries q .*

Proof. Let the cue anchor space be $\mathcal{C} := V$, and associate each memory entry with exactly one cue anchor:

$$\Gamma(m) := \{\pi(m)\}, \quad \forall m \in \mathcal{M}.$$

Let the primary abstraction space be trivial so that abstraction-based retrieval does not affect the result.

Define cue scoring such that

$$\text{TopK}_{c \in \mathcal{C}} s_C(q, c) = S(q),$$

yielding the initial retrieval

$$\mathcal{R}_0(q) = \{m \in \mathcal{M} \mid \pi(m) \in S(q)\}.$$

Define cue–cue traversal in Memora using the same similarity relation:

$$c \rightsquigarrow c' \iff \text{sim}(c, c') \geq \delta.$$

Applying L traversal steps retrieves exactly those memory entries whose associated cues lie in $\text{Nbr}_L^{\text{imp}}(S(q))$, hence

$$\mathcal{R}_L(q) = \mathcal{R}_{\text{KG}}^{\text{imp}}(q).$$

$\square$

Explicit KG retrieval. We now consider traditional knowledge-graph retrieval with explicit symbolic relations. Let $G = (V, E)$ be a knowledge graph, where V denotes entities and E denotes typed relations between entities. Each memory entry is attached to a graph element through a mapping

$$\pi : \mathcal{M} \rightarrow V \cup E.$$

Given a query q , KG retrieval selects a seed set $S(q) \subseteq V \cup E$ and retrieves memory entries associated with elements in the L -hop graph neighborhood:

$$\mathcal{R}_{\text{KG}}^{\text{exp}}(q) = \{m \in \mathcal{M} \mid \pi(m) \in \text{Nbr}_L(S(q))\}.$$

Theorem D.3 (Explicit KG Retrieval as an Extended Case of Memora). *For any explicit KG retriever $\mathcal{R}_{\text{KG}}^{\text{exp}}(q)$, there exists an extended Memora instantiation such that the multi-hop retrieval result $\mathcal{R}_L(q)$ produced by Memora equals $\mathcal{R}_{\text{KG}}^{\text{exp}}(q)$ for all queries q .*

Proof. Consider an extended Memora configuration in which cue anchors explicitly encode KG entities and relations by setting

$$\mathcal{C} := V \cup E, \quad \Gamma(m) := \{\pi(m)\}, \quad \forall m \in \mathcal{M}.$$

In addition, Memora is augmented with a cue–cue traversal relation that exactly mirrors the KG structure:

$$c \rightsquigarrow c' \iff (c, c') \in E.$$

This extension requires Memora to adopt the same relational assumptions as the underlying KG, namely that edges are explicitly defined and traversable.

Seed selection is performed through cue scoring such that

$$\text{TopK}_{c \in \mathcal{C}} s_C(q, c) = S(q),$$

yielding the initial retrieval

$$\mathcal{R}_0(q) = \{m \in \mathcal{M} \mid \pi(m) \in S(q)\}.$$

Since cue–cue traversal coincides exactly with KG edges, applying L traversal steps in Memora recovers the same L -hop neighborhood as $\text{Nbr}_L(S(q))$, and therefore

$$\mathcal{R}_L(q) = \mathcal{R}_{\text{KG}}^{\text{exp}}(q).$$

□

Interpretation. Explicit KG retrieval corresponds to an extended instantiation of Memora in which cue anchors are restricted to symbolic entities and relations, and traversal operations are constrained to follow predefined KG edges. This setting recovers classical KG behavior but requires Memora to inherit the same structural assumptions and construction costs as the KG. In contrast, the implicit KG case arises naturally within the base Memora design, where cue anchors and traversal relations can be learned or induced without explicit symbolic graphs.

D.3. Memora as a Strict Generalization: Expressivity

The special-case results above establish that flat RAG-style retrieval and KG-style seed-and-expand retrieval can be realized within Memora under suitable parameterizations. We next formalize a strictness result showing that Memora can represent retrieval behaviors that are not realizable by (i) flat top- k similarity retrieval over raw memory content and (ii) KG retrievers with a fixed single-attachment structure, under standard structural constraints.

Definition D.4 (Retrieval classes). A retrieval function maps a query to a subset of memory entries,

$$\mathcal{R} : \mathcal{Q} \rightarrow 2^{\mathcal{M}}.$$

We consider the following three retrieval classes.

1. **Flat top- k similarity retrieval.** There exists a single scoring function $s(q, m)$ such that, for every query q ,

$$\mathcal{R}(q) = \text{TopK}_k(\{s(q, m) : m \in \mathcal{M}\}),$$

and therefore $|\mathcal{R}(q)| = k$.

2. **KG seed-and-expand retrieval with fixed attachment.** There exists a fixed attachment map $\pi : \mathcal{M} \rightarrow V$ on a fixed graph $G = (V, E)$ such that, for every query q ,

$$\mathcal{R}(q) = \{m \in \mathcal{M} : \pi(m) \in \text{Nbr}_L(S(q))\},$$

where $S(q) \subseteq V$ is a query-dependent seed set and $\text{Nbr}_L(\cdot)$ denotes the L -hop neighborhood operator.

3. **Memora retrieval.** The retrieval function is realizable by Memora using primary abstractions $\alpha(m)$ and cue anchors $\Gamma(m)$, including the gated form

$$\mathcal{R}_\cap(q) := \{m \in \mathcal{M} : \alpha(m) \in A_q\} \cap \{m \in \mathcal{M} : \Gamma(m) \cap C_q \neq \emptyset\}, \quad (29)$$

where

$$A_q = \text{TopK}_{K_A}(\{s_A(q, a) : a \in \mathcal{A}\}), \quad C_q = \text{TopK}_{K_C}(\{s_C(q, c) : c \in \mathcal{C}\}).$$

Theorem D.5 (Strictness under mixed-key constraints). *There exists a Memora retrieval function $\mathcal{R}^*$ such that, for any fixed k and any fixed L , $\mathcal{R}^*$ cannot be realized by flat top- k similarity retrieval and cannot be realized by KG seed-and-expand retrieval with a fixed single-attachment map.*

Proof. We prove the theorem by giving an explicit construction of a retrieval function realizable by Memora but not by fixed top- k or fixed-attachment KG retrieval. The construction targets a retrieval behavior defined by the joint enforcement of two constraints: a coarse structural restriction induced by a primary abstraction, and a fine-grained selector induced by a cue anchor. Memora can realize such mixed-key constraints through intersection across its indexing spaces, whereas flat top- k retrievers and KG retrievers with fixed single-attachment are inherently unable to represent this joint selection under the stated constraints.

Step 1: Mixed-key target. Partition the memory corpus using two primary abstractions $\mathcal{A} = \{a^{(1)}, a^{(2)}\}$ and define

$$\mathcal{M}^{(1)} := \{m \in \mathcal{M} : \alpha(m) = a^{(1)}\}, \quad \mathcal{M}^{(2)} := \{m \in \mathcal{M} : \alpha(m) = a^{(2)}\}.$$

Fix a cue anchor $c^* \in \mathcal{C}$ appearing in both groups, and let

$$\mathcal{N}^{(1)} := \{m \in \mathcal{M}^{(1)} : c^* \in \Gamma(m)\}, \quad \mathcal{N}^{(2)} := \{m \in \mathcal{M}^{(2)} : c^* \in \Gamma(m)\}.$$

Define the target retrieval function

$$\mathcal{R}^*(q) := \mathcal{N}^{(1)},$$

and assume $|\mathcal{N}^{(1)}| > k$.

Step 2: Realizability within Memora. We show that the target retrieval function $\mathcal{R}^*$ is realizable by Memora. By definition, Memora supports retrieval predicates formed by the intersection of abstraction-level selection and cue-level selection. Consider the abstraction set $A_q = \{a^{(1)}\}$ and the cue set $C_q = \{c^*\}$. Since Memora allows independent query-conditioned selection over the abstraction space $\mathcal{A}$ and the cue space $\mathcal{C}$, there exist scoring functions s_A and s_C and finite cutoffs K_A, K_C such that these sets are selected.

Substituting these sets into the gated retrieval operator in Eq. (29) yields

$$\mathcal{R}_\cap(q) = \{m \in \mathcal{M} : \alpha(m) = a^{(1)}\} \cap \{m \in \mathcal{M} : c^* \in \Gamma(m)\} = \mathcal{R}^*(q).$$

Thus $\mathcal{R}^*$ lies within the class of retrieval functions realizable by Memora.

Step 3: Impossibility for flat top- k similarity retrieval. We show that $\mathcal{R}^*$ cannot be realized by any flat top- k similarity retriever. By definition, any such retriever is induced by a single real-valued scoring function $s : \mathcal{Q} \times \mathcal{M} \rightarrow \mathbb{R}$ and returns exactly k memory entries for every query:

$$|\mathcal{R}(q)| = k, \quad \forall q \in \mathcal{Q}. \quad (30)$$

In contrast, the target retrieval function $\mathcal{R}^*$ is defined as

$$\mathcal{R}^*(q) = \mathcal{N}^{(1)},$$

where $|\mathcal{N}^{(1)}| > k$ by construction. Therefore,

$$|\mathcal{R}^*(q)| > k. \quad (31)$$

Equations (30) and (31) immediately yield a contradiction: no flat top- k retriever can reproduce the output of $\mathcal{R}^*$ for this query.

More fundamentally, flat similarity retrieval induces a total preorder over $\mathcal{M}$ via $s(q, \cdot)$ and selects a prefix of fixed length k . Any retrieval predicate whose extension is not expressible as such a fixed prefix—independent of internal structure or semantic grouping—lies outside the expressive scope of flat top- k retrieval. Hence $\mathcal{R}^*$ is not realizable by flat similarity ranking.

Step 4: Impossibility for KG retrieval with fixed single-attachment. We now show that $\mathcal{R}^*$ cannot be realized by KG-style seed-and-expand retrieval under a fixed single-attachment map $\pi : \mathcal{M} \rightarrow V$.

For any such retriever, the retrieved set has the form

$$\mathcal{R}(q) = \{m \in \mathcal{M} : \pi(m) \in \text{Nbr}_L(S(q))\}, \quad (32)$$

where $\text{Nbr}_L(\cdot)$ denotes L -hop graph neighborhoods. Crucially, membership in $\mathcal{R}(q)$ depends *only* on the attachment $\pi(m)$ and the graph structure, and is therefore invariant to any memory attributes not encoded in $\pi(m)$.

In the constructed instance, memories in $\mathcal{N}^{(1)}$ and $\mathcal{N}^{(2)}$ share the same cue anchor c^* but differ in their primary abstractions $\alpha(m)$. Since π is fixed and single-valued, it cannot simultaneously encode both abstraction-level information and cue-level information without collapsing distinct semantic dimensions. As a result, there exist memories

$$m_1 \in \mathcal{N}^{(1)}, \quad m_2 \in \mathcal{N}^{(2)}$$

such that $\pi(m_1)$ and $\pi(m_2)$ lie in the same or overlapping graph neighborhoods whenever the cue signal c^* is reachable.

Consequently, any seed set $S(q)$ and radius L for which

$$m_1 \in \mathcal{R}(q)$$

necessarily implies

$$m_2 \in \mathcal{R}(q),$$

unless the graph or attachment map explicitly encodes the abstraction partition. This contradicts the definition of $\mathcal{R}^*$, which selects $\mathcal{N}^{(1)}$ while excluding $\mathcal{N}^{(2)}$.

Therefore, under a fixed single-attachment structure, KG neighborhood expansion cannot enforce the joint predicate

$$\alpha(m) = a^{(1)} \wedge c^* \in \Gamma(m),$$

and $\mathcal{R}^*$ is not realizable by KG retrieval.

Remark. The strictness result follows from mixed-key constraints: Memora can jointly enforce coarse structural scope through primary abstractions and fine-grained selection through cue anchors, a capability unavailable to flat similarity retrieval and KG retrieval with fixed single-attachment under standard assumptions. $\square$

Theorem D.6 (Efficiency gain from abstraction-first + cue-anchor ANN retrieval). *Assume that memories are partitioned into abstractions with expected bucket size B , so that $|\mathcal{A}| \approx N/B$, and that each abstraction has on average m cue anchors, yielding a total of $m|\mathcal{A}| \approx mN/B$ cue anchors indexed for retrieval. Under the variant in which query-time retrieval is performed via (1) an ANN lookup over abstractions and (2) an ANN lookup over cue anchors, without explicit intra-abstraction enumeration, the expected query-time cost of Memora satisfies*

$$T_{\text{Harmo}}(q) = O\left(\log\left(\frac{mN^2}{B^2}\right)\right).$$

In contrast, a flat ANN-based retriever that indexes all N memories incurs $T_{\text{RAG}}(q) = O(\log N)$ under the same index-family assumptions. Consequently, abstraction-first retrieval yields a multiplicative efficiency improvement of

$$\Omega\left(\frac{\log N}{2 \log N + \log m - 2 \log B}\right).$$

Proof. We upper bound the query-time cost of Memora by decomposing retrieval into two indexed lookups, and compare against a flat ANN baseline.

Stage 1: Abstraction selection. Memora queries an ANN index over abstractions $\mathcal{A}$. Since abstractions partition N memories into buckets of expected size B , we have $|\mathcal{A}| \approx N/B$. Under standard ANN index assumptions, querying an index of size $|\mathcal{A}|$ incurs expected cost

$$O(\log |\mathcal{A}|) = O\left(\log\left(\frac{N}{B}\right)\right).$$

Stage 2: Cue-anchor retrieval. Memora then performs an ANN query over the cue-anchor index. If each abstraction has on average m cue anchors, then the cue-anchor index size is

$$|\mathcal{U}| \approx m|\mathcal{A}| \approx \frac{mN}{B}.$$

Thus the cue-anchor query incurs expected cost

$$O(\log |\mathcal{U}|) = O\left(\log\left(\frac{mN}{B}\right)\right).$$

By assumption of this variant, retrieved cue anchors provide direct references to associated memories, so there is no additional intra-abstraction enumeration term.

Total cost. Summing the two stages yields

$$\begin{aligned} T_{\text{Harmo}}(q) &= O\left(\log\left(\frac{N}{B}\right)\right) + O\left(\log\left(\frac{mN}{B}\right)\right) \\ &= O\left(\log\left(\frac{N}{B} \cdot \frac{mN}{B}\right)\right) \\ &= O\left(\log\left(\frac{mN^2}{B^2}\right)\right). \end{aligned}$$

Flat ANN baseline. A flat ANN-based retriever performs a single ANN query over N indexed memories, incurring expected query time

$$T_{\text{RAG}}(q) = O(\log N)$$

under the same index-family assumptions.

Improvement (normalized form). Define the multiplicative efficiency improvement as $T_{\text{RAG}}(q)/T_{\text{Harmo}}(q)$. Substituting the bounds gives

$$\frac{T_{\text{RAG}}(q)}{T_{\text{Harmo}}(q)} = \Omega\left(\frac{\log N}{\log\left(\frac{mN^2}{B^2}\right)}\right).$$

Expanding the denominator,

$$\log\left(\frac{mN^2}{B^2}\right) = 2\log N + \log m - 2\log B,$$

so

$$\frac{T_{\text{RAG}}(q)}{T_{\text{Harmo}}(q)} = \Omega\left(\frac{\log N}{2\log N + \log m - 2\log B}\right).$$

which proves the stated improvement bound. $\square$

Analysis. Conditions under which the efficiency improvement exceeds unity.

Let

$$\text{Imp}(N, B, m) = \frac{T_{\text{RAG}}(q)}{T_{\text{Harmo}}(q)} \approx \frac{\log N}{\log\left(\frac{mN^2}{B^2}\right)}.$$

A sufficient condition for $\text{Imp}(N, B, m) > 1$ is

$$\log N > \log\left(\frac{mN^2}{B^2}\right) \iff N > \frac{mN^2}{B^2} \iff B^2 > mN.$$

Equivalently, in normalized form,

$$\text{Imp}(N, B, m) > 1 \quad \text{whenever} \quad 2 + \frac{\log m}{\log N} - 2\frac{\log B}{\log N} < 1 \iff \frac{\log B}{\log N} > \frac{1}{2} \left(1 + \frac{\log m}{\log N}\right).$$

In particular, if m is constant (or grows subpolynomially) and $B = \Omega(N^{1/2+\epsilon})$ for any $\epsilon > 0$, then $\text{Imp}(N, B, m) > 1$ for sufficiently large N .

Remark. In typical memory systems, $B^2 > mN$ is a strong requirement; when it does not hold, both approaches remain logarithmic and the advantage of abstraction-first retrieval should be interpreted primarily as a constant-factor gain due to operating over smaller indices (and, in practice, fewer distance computations and better cache locality).

Implication. Primary abstraction provides a principled *search space factorization*: the retrieval process first narrows the search space using stable, coarse-grained concepts, and then applies cue anchors to recover fine-grained precision within the selected regions. Flat RAG is recovered as a degenerate case when $B = 1$ (each memory forms its own abstraction) or when abstraction selection is disabled. KG retrieval is recovered when cue anchors correspond to symbolic graph elements and candidate expansion follows graph adjacency, as established in Theorems D.1 and D.3.

D.4. Summary

The Memora framework defines a general class of structured retrieval mechanisms based on (i) canonical organization via primary abstraction and (ii) flexible access via cue anchors, optionally enhanced with multi-hop traversal. We formally showed that: (i) traditional RAG is a degenerate special case (identity cues, no abstraction), (ii) KG retrieval is also a special case (symbolic cues + graph expansion), and (iii) Memora can represent richer mixed-key retrieval behaviors while enabling principled efficiency improvements through abstraction-first scoping.

E. Case Study

In this section, we present case studies demonstrating how MEMORA achieves superior performance compared to Mem0 and RAG. To isolate the benefits of our memory structure, we utilize MEMORA with a standard semantic retriever, and showcase the factual memories, thereby highlighting how the harmonic representation itself enhances memory management and retrieval.

E.1. Case Study 1

In this example (Table 8), MEMORA demonstrates superior memory retrieval precision. This success is attributed to MEMORA’s index-value representation, which decouples the navigation layer from the raw data. While traditional RAG often suffers from semantic drift and Mem0 can lose granularity through over-summarization, MEMORA’s indices serve as a structured guide to the memory space. This allows the system to pinpoint specific entities while preserving the original richness and contextual meaning of the memory items.

Table 8. Case 1 and answers generated from three systems

Question	What did Mel and her kids paint in their latest project in July 2023?
Reference Answer	A sunset with a palm tree
RAG Answer	✗ A rainbow flag mural
Mem0 Answer	✗ A painting similar to their last one
MEMORA Answer	✓ Sunset scene with a palm tree and flowers

Table 9. Comparative analysis of top memories retrieved for Case 1. (part 1)

Method	Retrieved Memories / Contextual Evidence
MEMORA	Recent painting by Melanie and kids <i>Value:</i> Melanie’s latest painting with the kids is a sunset scene featuring a palm tree and vibrant flowers against a sunset sky. <i>Cues:</i> ‘Melanie sunset painting’, ‘Palm tree art’, ‘Kids vibrant flowers’
	Melanie’s work in progress and summer plans <i>Value:</i> Melanie is currently working on a project and is doing her best to complete it, her kids are excited about summer break, and they are thinking about going camping next month. <i>Cues:</i> ‘Melanie current project’, ‘Kids summer break’, ‘Family camping plans’
	Melanie’s kids enjoying pottery making <i>Value:</i> Melanie’s kids loved making pottery and were very excited to get their hands dirty and create something with clay. <i>Cues:</i> ‘Kids pottery making’, ‘Clay art activity’
	Melanie’s recent family painting activity <i>Value:</i> Melanie and her kids have just finished another painting similar to their last one. <i>Cues:</i> ‘Melanie family painting’, ‘Kids collaborative artwork’
	Melanie’s creative projects with kids <i>Value:</i> Melanie engages in painting with kids, focusing especially on nature-inspired themes. <i>Cues:</i> ‘Melanie nature painting’, ‘Kids art engagement’

Table 10. Comparative analysis of top memories retrieved for Case 1. (continued)

Method	Retrieved Memories / Context
Mem0	Melanie and children recently did a painting project last weekend. Melanie and her kids recently finished another painting similar to their last one. Melanie and children enjoy painting together, especially nature-inspired art. Melanie has been painting to keep busy. Melanie took her kids to a pottery workshop last Friday. Melanie helped with a painting that highlights the beauty of nature. Melanie is feeling inspired by autumn and planning a few paintings.
RAG	Context Fragment: 1:33 pm on 25 August, 2023 — Caroline: Finding a community where I’m accepted... Stuff like this mural are really special to me! ... 1:33 pm on 25 August, 2023 — Melanie: Caroline, glad you found a supportive community! ... 1:33 pm on 25 August, 2023 — Caroline: The rainbow flag mural is important to me as it reflects the courage and strength of the trans community. The eagle symbolizes freedom and pride... ... 1:33 pm on 25 August, 2023 — Melanie: I’m in awe of your courage as a trans person. Have you made any more art lately?

E.2. Case Study 2

In this example (Table 11), MEMORA demonstrates a robust capacity for information aggregation, correctly synthesizing disparate facts, the initial ownership of a dog and cat, followed by the later addition of a second cat named Bailey, into a single, comprehensive response. Unlike baseline methods that often retrieve fragmented memory fragments or lose connections, MEMORA effectively links related entities across non-contiguous parts of the dialogue context.

Table 11. Case Study 2 and answers generated from three systems

Question	What pets does Melanie have?
Reference Answer	Two cats and a dog
RAG Answer	✗ A cat named Oliver and another cat named Bailey
Mem0 Answer	✗ Luna and Oliver
MEMORA Answer	✓ Dog and two cats (Luna, Oliver, Bailey)

E.3. Case Study 3

In this example (Table 14), a scrutiny of the retrieved evidence reveals that while baseline methods identify the correct topical domain, they fail to capture the discriminative details required for an accurate response. The RAG retrieval is too broad and lacks relevance; it becomes anchored to a dense, irrelevant dialogue fragment about a “colorful bowl” from a separate project, illustrating how raw context windows are easily distracted by high-signal but incorrect semantic clusters. Meanwhile, Mem0 produces a set of isolated, low-entropy facts, such as “the kids enjoyed making things with clay”, which, while factually true, are too fragmented and generic to support the specific query. By contrast, MEMORA successfully preserves the fine-grained entity binding between the “kids’ pottery” and the “dog-face cup.” Its index-value architecture prevents the information decay seen in Mem0 and the noise contamination seen in RAG, ensuring that specific attributes remain intact within the retrieved memory.

Table 12. Comparison of top retrieved memories for Case Study 2. (part 1)

Method	Retrieved Memories / Contextual Evidence
MEMORA	Melanie’s pets and inquiries about pets <i>Value:</i> Melanie has a dog and a cat as pets. Melanie has two pets named Luna and Oliver. Melanie asked Caroline if she has any pets during their conversation. <i>Cues:</i> ‘Melanie pets’, ‘Melanie pet names’, ‘Melanie conversation about pets’
	Melanie’s agreement on pets <i>Value:</i> Melanie agrees that pets bring joy and comfort. <i>Cues:</i> ‘Melanie pets joy’, ‘Melanie pets comfort’
	Melanie’s pets behavior <i>Value:</i> Melanie’s pets, Luna and Oliver, are described as sweet and playful and they really liven up the house. <i>Cues:</i> ‘Melanie pets behavior’, ‘Luna playful’, ‘Oliver playful’
	Pets’ effect on Melanie’s family <i>Value:</i> Melanie states that their dog and cat brighten up their day and always make them smile. <i>Cues:</i> ‘Melanie pets family effect’, ‘Melanie pets brighten day’
	Melanie’s cat Bailey addition <i>Value:</i> Melanie mentions that they have another cat named Bailey. <i>Cues:</i> ‘Melanie cat Bailey’

Table 13. Comparison of top retrieved memories for Case Study 2. (continued)

Method	Retrieved Memories / Context
Mem0	Melanie has kids. Melanie loves painting animals, especially horses. User knows Melanie. Melanie has been painting to keep busy. Caroline finds joy in having pets. Melanie paints horses. Name is Melanie.
RAG	Transcript Fragment: 3:31 pm on 23 August, 2023 — Caroline: ...And yup, I do— Oscar, my guinea pig. He’s been great. How are your pets? ... 3:31 pm on 23 August, 2023 — Melanie: Yeah, it’s normal to be both excited and nervous with a big decision. And thanks for asking, they’re good— we got another cat named Bailey too. Here’s a pic of Oliver. Can you show me one of Oscar?...

Table 14. Case Study 3 and answers generated from three systems.

Question	What kind of pot did Mel and her kids make with clay?
Reference Answer	A cup with a dog face on it
RAG Answer	✗ A colorful bowl with various colors and patterns
Mem0 Answer	✗ Black and white designed bowl
MEMORA Answer	✓ A cup with a dog face

Table 15. Comparison of top retrieved memories for Case Study 3. (part 1)

Method	Retrieved Memories / Contextual Evidence
MEMORA	Melanie's kids enjoying pottery making <i>Value:</i> Melanie's kids loved making pottery and were very excited to get their hands dirty and create something with clay. <i>Cues:</i> 'Melanie kids pottery', 'Kids clay crafting'
	Melanie's feelings about clay <i>Value:</i> Melanie finds clay to be incredible and it brings her a lot of joy. <i>Cues:</i> 'Melanie clay appreciation', 'Melanie joy from clay'
	Melanie's recent activity with kids <i>Value:</i> Melanie took her kids to a park yesterday, where they had fun exploring and playing. <i>Cues:</i> 'Melanie park visit', 'Kids outdoor play'
	Melanie's kids pottery finished pieces <i>Value:</i> The kids created pottery finished pieces, including a cup with a dog face on it. <i>Cues:</i> 'Kids pottery artwork', 'Pottery cup dog face'
	Melanie's pottery as a creative and therapeutic outlet <i>Value:</i> Melanie signed up for a pottery class, which she considers therapeutic and allows her to express herself and be creative. Melanie loves that pottery is both a creative outlet and a form of therapy. Melanie uses pottery as a means for self-expression and to find peace. <i>Cues:</i> 'Melanie pottery therapy', 'Pottery creative expression'

Table 16. Comparison of top retrieved memories for Case Study 3. (continued)

Method	Retrieved Memories / Context
Mem0	The kids enjoyed making things with clay. Melanie took her kids to a pottery workshop last Friday. Melanie recently finished a pottery project. Pottery is a huge part of Melanie's life and helps her express her emotions. Melanie is proud of her pottery project and had a great experience making it. Enjoyed making pots with kids. The kids loved making something with clay.
RAG	Transcript Fragment: 1:50 pm on 17 August, 2023 — Melanie: FYI, I finished another pottery project— want to see a pic? ... 1:50 pm on 17 August, 2023 — Caroline: That bowl is awesome, Mel! What gave you the idea for all the colors and patterns? ... 1:50 pm on 17 August, 2023 — Melanie: Thanks, Caroline! I'm obsessed with those, so I made something to catch the eye...

Table 17. Sensitivity to the memory update threshold on LoCoMo.

Threshold	Multi	Temp	Open	Single	Overall	Avg. updates	Update ratio
No update	0.727	0.801	0.542	0.844	0.795	0	0.0%
0.8 (default)	0.780	0.813	0.510	0.836	0.801	88	21.0%
0.6	0.755	0.801	0.552	0.842	0.799	299	68.2%

Table 18. Update behavior as memories accumulate (threshold = 0.8).

Memories accumulated	Avg. new	Avg. update	Update ratio
0–76	77.7	16.0	17.1%
77–155	79.5	22.7	22.2%
156–234	74.9	21.4	22.2%
235–318	82.1	23.4	22.2%
319+	66.4	13.1	16.5%

F. Memory Update Analysis

Threshold sensitivity. MEMORA’s update mechanism merges a newly extracted memory into an existing entry when their primary abstractions exceed a semantic-similarity threshold. We use a default threshold of 0.80, chosen to merge only entries that are clearly redundant while preventing over-aggressive consolidation.

To isolate the effect of the update threshold from other design choices, we ablate three settings using semantic retrieval with primary abstractions only (no cue anchors or episodic memory); absolute scores are therefore lower than the best results reported in Table 1.

Table 17 shows that the default 0.8 achieves the highest overall LLM-as-a-Judge score. Lowering the threshold to 0.6 triggers $3.4\times$ more updates with no quality gain, indicating over-consolidation: semantically distinct entries get merged into the same memory. Disabling updates entirely reduces overhead but hurts quality, particularly on multi-hop questions (0.727 vs. 0.780), where consolidation helps resolve redundant or fragmented entries that would otherwise pull the retriever in conflicting directions.

Memory growth and update. A natural concern with similarity-based consolidation is whether the update rate grows pathologically as the memory store fills up. On average, each LoCoMo conversation produces 432 candidate memories, of which 344 are inserted as new entries and 88 trigger an update (an overall update ratio of 21.0%). To examine scaling behavior we bucket each memory-construction event by the number of memories already present at the time of construction; Table 18 reports the per-bucket update ratio. The ratio remains in a narrow 16.5–22.2% band across all buckets rather than increasing exponentially, indicating that consolidation cost scales linearly with memory size and does not become a bottleneck as the memory store grows.